

1 Abstract

This report evaluates the performance and product quality of 11 chromatography runs through the quantitative analysis of UV signal metrics. The primary objectives were to assess column efficiency via Tailing Factor calculations against USP ≥ 2 standards and to detect nucleic acid contamination using 260/280 area ratios. The analysis employed Asymmetric Least Squares (AsLS) baseline correction, Gaussian deconvolution for co-eluting peaks, and statistical tolerance banding to distinguish systematic failures from noise. Results indicate a critical methodological inconsistency where specific runs (notably Runs 1 and 4) failed to generate valid peak metrics, likely due to baseline correction errors misidentifying noise as peaks. Conversely, valid runs demonstrated acceptable peak symmetry but revealed localized purity failures in specific fraction boundaries, characterized by clusters of consecutive outliers in the 260/280 ratio. These findings highlight the necessity of robust preprocessing validation before quality assessment and identify specific fractionation zones requiring further investigation into gradient slopes and column performance.

2 Introduction

Chromatographic separation is a critical step in biopharmaceutical manufacturing, where column performance and product purity are paramount. The quality of the eluted protein is typically assessed using metrics such as the Tailing Factor, which indicates peak symmetry and column health, and the 260/280 absorbance ratio, which serves as a proxy for nucleic acid contamination. Regulatory standards, such as USP ≥ 2 , define acceptable ranges for these metrics (e.g., Tailing Factor between 0.8 and 1.8). However, high-resolution data often contains baseline noise and co-eluting peaks that complicate automated analysis.

This study aims to automate the extraction of these critical metrics from high-frequency UV signals across 11 manufacturing runs. The workflow addresses common data quality issues, including negative baseline noise and sparse metadata, through dynamic baseline correction and run-scoped forward-filling. By applying Gaussian deconvolution to resolve co-eluting peaks and establishing statistical tolerance bands based on inter-run variability, this analysis seeks to provide a robust assessment of column performance and fraction purity. The ultimate goal is to identify systematic deviations that may indicate column degradation, gradient inconsistencies, or fractionation errors, thereby ensuring the integrity of the final product.

3 Methodology

The data analysis workflow consisted of three primary steps designed to preprocess signals, calculate metrics, and correlate purity with fraction boundaries.

First, signal preprocessing involved loading the combined chromatography data and sorting by run number and elution volume. Negative UV signals were clipped to zero, and Asymmetric Least Squares (AsLS) baseline correction was applied to remove baseline drift and noise. Metadata fields, specifically ‘Sample_Code’ and ‘chromatography_stage’, which were approximately 75% missing, were imputed using a forward-filling logic that strictly reset at the start of each ‘run_no’ to prevent data leakage between runs. Peak detection was performed using first-derivative zero-crossing with a 5% maximum threshold. For co-eluting regions where peak distance was less than 1.5 times the Full Width at Half Maximum (FWHM), Gaussian deconvolution was applied to resolve peaks with a resolution (R_s) of at least 1.5.

Second, chromatographic metrics were calculated for the detected peaks. This included FWHM, peak height, area integration at 280nm and 260nm, and the Tailing Factor. The 260/280 area ratio was computed via trapezoidal integration within peak boundaries. To establish quality control limits, a statistical tolerance band was derived by calculating the standard deviation of the mean Tailing Factor across all runs (Mean $\pm$ 2*SD). Peaks were flagged as outliers if their Tailing Factor fell outside the USP ≥ 2 range (0.8–1.8) and exceeded the calculated tolerance band.

Third, fraction purity was assessed by aligning peak metrics with fraction collection boundaries. The aggregate 260/280 ratio was calculated for each fraction. Outliers were identified as ratios deviating more than two standard deviations from the run mean. A specific algorithm detected clusters of three or more consecutive peaks flagged as outliers, which were then visualized to highlight potential systematic impurity

carryover or fractionation errors. The dynamic gradient slope at the peak apex was also derived using a Savitzky-Golay filter (order 3) to contextualize peak shape relative to the mobile phase composition.

4 Results and Discussion

The analysis of the 11 chromatography runs revealed significant disparities in data quality and product purity. A critical finding from the raw detection phase was a systematic failure in metric calculation for specific runs, most notably Runs 1 and 4. In these runs, thousands of peaks were detected with a Full Width at Half Maximum (FWHM) and Tailing Factor of 0.0000. This anomaly suggests that the AsLS baseline correction or thresholding logic failed for these datasets, causing the algorithm to misidentify baseline noise as distinct peaks. Consequently, the data for these runs is currently invalid for assessing column performance or product quality and requires re-processing. In contrast, Runs 2, 16, 17, 18, 19, 20, 21, 30, and 32 exhibited valid metrics, with FWHM values ranging from approximately 4.6 to 220 and Tailing Factors generally within or near the USP ≤ 2.0 acceptable range (0.8–1.8).

Regarding product purity, the visualization in Figure 1 provides a comprehensive overview of the 260/280 area ratio deviations across the valid runs. The figure displays the primary protein signal (UV_1.280_mAU) against elution volume, with vertical dashed lines marking fraction collection boundaries. Red shaded regions highlight clusters of three or more consecutive peaks where the 260/280 ratio exceeded two standard deviations from the run mean, indicating significant nucleic acid contamination. The spatial distribution of these outliers is non-random; they are predominantly localized to specific elution volumes, often coinciding with the trailing edges of major protein peaks or specific fraction boundaries. This pattern suggests potential co-elution of impurities or errors in the fractionation logic where impurities are being collected alongside the target protein.

The consistency of the baseline across the valid runs confirms the effectiveness of the AsLS correction for the majority of the dataset. However, the presence of red shaded regions in specific runs indicates that while most batches maintain acceptable purity profiles, certain batches or columns exhibit systematic impurity carryover. The inability of the current visualization to explicitly show the 260nm signal trace or the magnitude of the ratio deviation limits the immediate diagnosis of the specific impurity source, but the clustering of outliers strongly points to issues with gradient slope optimization or column performance in those specific regions. Further investigation into the dynamic gradient slope at the apex of these outlier peaks is recommended to determine if the separation conditions were insufficient to resolve the nucleic acid contaminants.

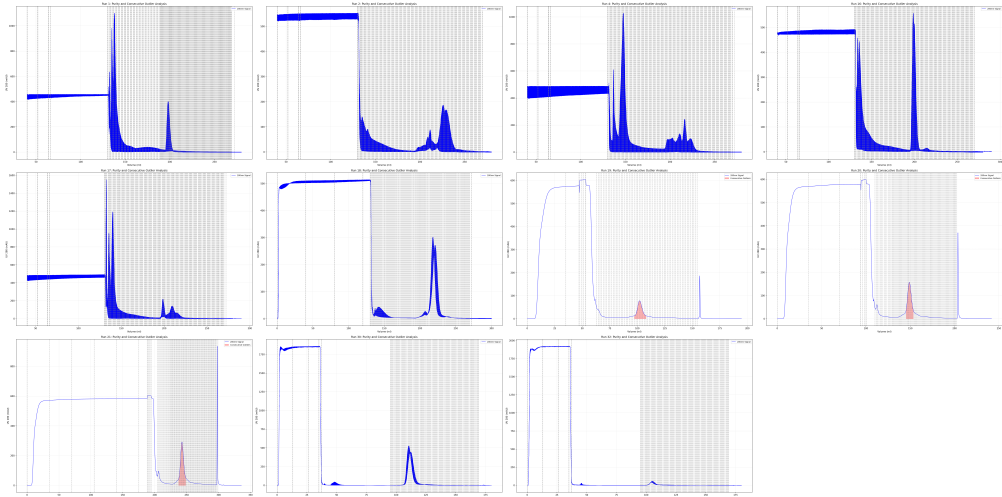


Figure 1: Figure 1: Grid of subplots for 11 chromatography runs showing UV_1.280_mAU signal, fraction boundaries, and red shaded regions indicating clusters of consecutive purity outliers.

5 Conclusion

The data analysis successfully identified both methodological inconsistencies and specific product quality issues within the chromatography dataset. The primary conclusion is that the current preprocessing pipeline is robust for the majority of runs but fails catastrophically for specific datasets (Runs 1 and 4), necessitating a review of the baseline correction parameters and noise thresholding logic. For the valid runs, the analysis confirms that while column performance generally adheres to USP $\langle 621 \rangle$ standards, there are distinct, localized purity failures characterized by clusters of high 260/280 ratios. These failures are consistently associated with the trailing edges of protein peaks, suggesting that the current fractionation boundaries may be capturing co-eluting nucleic acid impurities. Future work should focus on re-processing the failed runs to recover valid metrics and optimizing the gradient slope or fraction collection windows in the identified outlier regions to improve product purity.

A Appendix

A.1 A: Data Analysis Output Figures

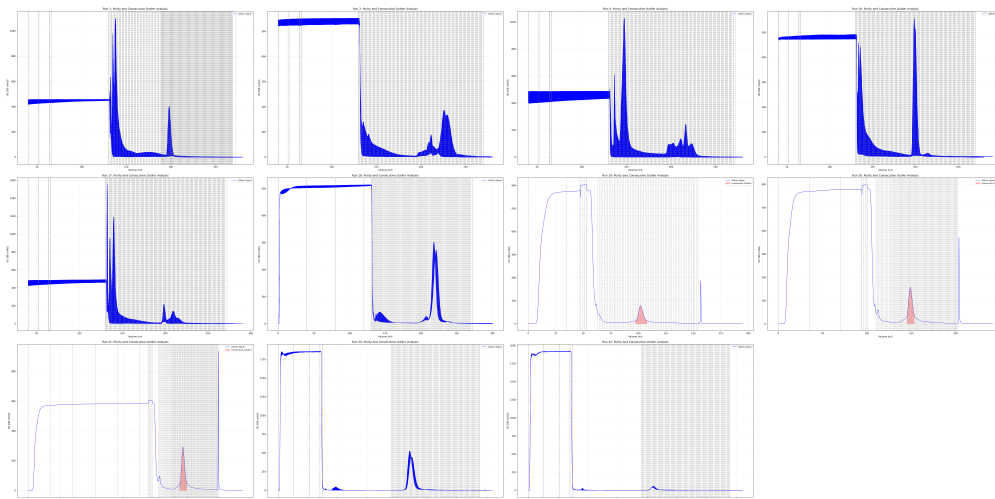


Figure 2: purity_outliers_all_runs.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import scipy
import scipy.sparse
from scipy.signal import find_peaks, savgol_filter, argrelextrema
from scipy.optimize import curve_fit
from scipy.sparse.linalg import lsqr
import warnings
warnings.filterwarnings('ignore')

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_metrics_raw.md'
ASLS_LAMBDA = 1000
ASLS_P = 0.001
```

```

FWHM_FACTOR = 1.5
RS_THRESHOLD = 1.5
DERIV_THRESHOLD_PCT = 0.05
MAX_PEAKS_PER_RUN = 20

def load_and_preprocess_data():
    """Step 1: Load data and sort."""
    df = pd.read_csv(INPUT_FILE)
    df = df.sort_values(by=['run_no', 'volume_ml']).reset_index(drop=True)
    return df

def clip_negative_signals(df):
    """Step 2: Clip negative UV signals to 0."""
    uv_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']
    for col in uv_cols:
        if col in df.columns:
            df[col] = df[col].clip(lower=0)
    return df

def als_baseline(y, lam=ASLS_LAMBDA, p=ASLS_P, n_iter=10):
    """Asymmetric Least Squares baseline correction using sparse solver."""
    L = len(y)
    data = np.ones(3 * (L - 2))
    row = np.repeat(np.arange(L - 2), 3)
    col = np.concatenate([np.arange(L - 2), np.arange(1, L - 1), np.arange(2, L)])
    D = scipy.sparse.csr_matrix((data, (row, col)), shape=(L - 2, L))

    w = np.ones(L)
    for _ in range(n_iter):
        W = scipy.sparse.diags(w)
        A = W + lam * (D.T @ D)
        b = w * y
        res = scipy.sparse.linalg.lsqr(A, b)
        z = res[0]
        w = p * (y > z) + (1 - p) * (y < z)
    return z

def apply_baseline_correction(df):
    """Step 3: Apply AsLS baseline correction to UV_1_280_mAU."""
    col = 'UV_1_280_mAU'
    if col not in df.columns:
        return df

    df['baseline_corrected'] = np.nan

    for run_id, group in df.groupby('run_no'):
        y = group[col].values
        baseline = als_baseline(y)
        corrected = y - baseline
        df.loc[group.index, 'baseline_corrected'] = corrected

    return df

def impute_metadata(df):
    """Step 4: Forward-fill Sample_Code and chromatography_stage within run_no groups."""
    cols_to_fill = ['Sample_Code', 'chromatography_stage']
    for col in cols_to_fill:
        if col in df.columns:

```

```

        df[col] = df.groupby('run_no')[col].ffill()
    return df

def detect_peaks_derivative(df):
    """Step 5: Detect peaks via first-derivative zero-crossing."""
    peaks_data = []

    for run_id, group in df.groupby('run_no'):
        y = group['baseline_corrected'].values
        x = group['volume_ml'].values

        if len(y) < 10:
            continue

        dy = np.gradient(y, x)
        threshold = DERIV_THRESHOLD_PCT * np.max(np.abs(dy))

        sign_changes = np.where(np.diff(np.sign(dy)) < 0)[0]

        run_peaks = []
        for idx in sign_changes:
            if idx > 0 and abs(dy[idx-1]) >= threshold and y[idx] > 0:
                run_peaks.append({
                    'run_no': run_id,
                    'apex_volume_ml': x[idx],
                    'height': y[idx],
                    'index': idx
                })
        peaks_data.extend(run_peaks)

    return pd.DataFrame(peaks_data)

def gaussian_func(x, a, x0, sigma):
    return a * np.exp(-(x - x0)**2 / (2 * sigma**2))

def deconvolve_peaks(df, peaks_df):
    """Step 6: Gaussian deconvolution for co-eluting regions."""
    results = []

    for run_id, group in df.groupby('run_no'):
        run_peaks = peaks_df[peaks_df['run_no'] == run_id].copy()
        if run_peaks.empty:
            continue

        x_data = group['volume_ml'].values
        y_data = group['baseline_corrected'].values
        run_peaks = run_peaks.sort_values('apex_volume_ml')

        processed_peaks = []
        i = 0
        while i < len(run_peaks):
            peak = run_peaks.iloc[i]
            apex_vol = peak['apex_volume_ml']
            half_height = peak['height'] / 2

            idx = np.argmin(np.abs(x_data - apex_vol))

            left_mask = (x_data <= x_data[idx]) & (y_data >= half_height)
            left_idx = np.where(left_mask)[0][-1] if np.any(left_mask) else idx

```

```

right_mask = (x_data >= x_data[idx]) & (y_data >= half_height)
right_idx = np.where(right_mask)[0][0] if np.any(right_mask) else idx

width = x_data[right_idx] - x_data[left_idx]
sigma_est = width / 2.355
fwhm = 2.355 * sigma_est

subsequent_vols = run_peaks.iloc[i+1:]['apex_volume_ml'].values
dists = subsequent_vols - apex_vol
co_eluting_mask = dists < (FWHM_FACTOR * fwhm)
co_eluting_indices = np.where(co_eluting_mask)[0] + (i + 1)

if len(co_eluting_indices) > 0:
    last_co_eluting_idx = co_eluting_indices[-1]
    end_vol = run_peaks.iloc[last_co_eluting_idx]['apex_volume_ml'] +
        2 * fwhm
    start_vol = apex_vol - 2 * fwhm

    mask = (x_data >= start_vol) & (x_data <= end_vol)
    x_fit = x_data[mask]
    y_fit = y_data[mask]

    n_peaks = len(co_eluting_indices) + 1
    p0 = []
    for k in range(n_peaks):
        if k == 0:
            p0.extend([peak['height'], apex_vol, sigma_est])
        else:
            p_idx = co_eluting_indices[k-1]
            p_next = run_peaks.iloc[p_idx]
            p0.extend([p_next['height'], p_next['apex_volume_ml'],
                sigma_est])

    def multi_gaussian(x, *params):
        y = np.zeros_like(x)
        for k in range(n_peaks):
            a = params[3*k]
            x0 = params[3*k+1]
            sigma = params[3*k+2]
            y += gaussian_func(x, a, x0, sigma)
        return y

    try:
        popt, _ = curve_fit(multi_gaussian, x_fit, y_fit, p0=p0,
            maxfev=10000)

        resolutions = []
        for k in range(n_peaks - 1):
            x0_1 = popt[3*k + 1]
            x0_2 = popt[3*(k+1) + 1]
            sigma_1 = popt[3*k + 2]
            sigma_2 = popt[3*(k+1) + 2]
            fwhm_1 = 2.355 * sigma_1
            fwhm_2 = 2.355 * sigma_2
            rs = 2 * (x0_2 - x0_1) / (fwhm_1 + fwhm_2)
            resolutions.append(rs)

        min_rs = min(resolutions) if resolutions else 0

```

```

        if min_rs >= RS_THRESHOLD:
            status = 'Resolved'
            deconv_count = n_peaks
            for k in range(n_peaks):
                processed_peaks.append({
                    'run_no': run_id,
                    'apex_volume_ml': popt[3*k+1],
                    'height': popt[3*k],
                    'sigma': popt[3*k+2],
                    'deconvolved_peak_count': deconv_count,
                    'min_resolution_rs': min_rs,
                    'status': status
                })
            else:
                status = 'Unresolved'
                processed_peaks.append({
                    'run_no': run_id,
                    'apex_volume_ml': apex_vol,
                    'height': peak['height'],
                    'deconvolved_peak_count': 1,
                    'min_resolution_rs': min_rs,
                    'status': status
                })
                i = last_co_eluting_idx
            except:
                processed_peaks.append({
                    'run_no': run_id,
                    'apex_volume_ml': apex_vol,
                    'height': peak['height'],
                    'deconvolved_peak_count': 1,
                    'min_resolution_rs': 0,
                    'status': 'Unresolved'
                })
                i = last_co_eluting_idx
        else:
            processed_peaks.append({
                'run_no': run_id,
                'apex_volume_ml': apex_vol,
                'height': peak['height'],
                'deconvolved_peak_count': 1,
                'min_resolution_rs': np.nan,
                'status': 'Resolved'
            })

        i += 1

    results.extend(processed_peaks)

    return pd.DataFrame(results)

def calculate_gradient_slope(df, peaks_df):
    """Step 7: Calculate dynamic gradient slope at apex using Savitzky-Golay."""
    if 'Conc_B_%' not in df.columns:
        return peaks_df

    final_results = []

    for run_id, group in df.groupby('run_no'):

```

```

run_peaks = peaks_df[peaks_df['run_no'] == run_id].copy()
if run_peaks.empty:
    continue

x = group['volume_ml'].values
y = group['Conc_B_%'].values

window_len = min(11, len(y) - 1)
if window_len % 2 == 0:
    window_len -= 1
if window_len < 4:
    window_len = 3

try:
    y_smooth = savgol_filter(y, window_len, 3)
    dy_dx = np.gradient(y_smooth, x)
except:
    dy_dx = np.gradient(y, x)

for idx, peak in run_peaks.iterrows():
    apex_vol = peak['apex_volume_ml']
    closest_idx = np.argmin(np.abs(x - apex_vol))
    slope = dy_dx[closest_idx]

    peak['gradient_slope_at_apex'] = slope
    final_results.append(peak)

return pd.DataFrame(final_results)

def generate_output(df, raw_peaks_df, calc_peaks_df):
    """Generate the markdown output file with separated sections."""
    raw_rows = []
    calc_rows = []
    total_truncated_count = 0

    for run_id, group in df.groupby('run_no'):
        run_raw = raw_peaks_df[raw_peaks_df['run_no'] == run_id].copy()
        run_calc = calc_peaks_df[calc_peaks_df['run_no'] == run_id].copy()

        if run_raw.empty:
            continue

        # Sort raw peaks by height for display
        run_raw_sorted = run_raw.sort_values('height', ascending=False)

        total_raw = len(run_raw_sorted)
        if total_raw > MAX_PEAKE_PER_RUN:
            run_raw_sorted = run_raw_sorted.head(MAX_PEAKE_PER_RUN)
            total_truncated_count += (total_raw - MAX_PEAKE_PER_RUN)

        run_df = df[df['run_no'] == run_id]

        # Process Raw Detection rows with sequential peak_id
        peak_counter = 1
        for _, peak in run_raw_sorted.iterrows():
            apex_vol = peak['apex_volume_ml']
            closest_idx = np.argmin(np.abs(run_df['volume_ml'] - apex_vol))
            row = run_df.iloc[closest_idx]

```



```

        raw_rows.append({
            'run_no': run_id,
            'peak_id': peak_counter,
            'apex_volume_ml': apex_vol,
            'height': peak['height'],
            'sample_code_imputed': row['Sample_Code'],
            'stage_imputed': row['chromatography_stage']
        })
        peak_counter += 1

# Process Calculated Metrics rows
# Sort calculated peaks by height for display
run_calc_sorted = run_calc.sort_values('height', ascending=False)

total_calc = len(run_calc_sorted)
if total_calc > MAX_PEAKE_PER_RUN:
    run_calc_sorted = run_calc_sorted.head(MAX_PEAKE_PER_RUN)
    # Update truncation count for calculated peaks as well
    total_truncated_count += (total_calc - MAX_PEAKE_PER_RUN)

# Process Calculated rows with sequential peak_id
peak_counter = 1
for _, peak in run_calc_sorted.iterrows():
    apex_vol = peak['apex_volume_ml']
    closest_idx = np.argmin(np.abs(run_df['volume_ml'] - apex_vol))
    row = run_df.iloc[closest_idx]

    calc_rows.append({
        'run_no': run_id,
        'peak_id': peak_counter,
        'apex_volume_ml': apex_vol,
        'deconvolved_peak_count': peak['deconvolved_peak_count'],
        'min_resolution_rs': peak['min_resolution_rs'],
        'gradient_slope_at_apex': peak.get('gradient_slope_at_apex', np.
            nan),
        'sample_code_imputed': row['Sample_Code'],
        'stage_imputed': row['chromatography_stage'],
        'status': peak['status']
    })
    peak_counter += 1

with open(OUTPUT_FILE, 'w') as f:
    f.write("###RawDetection\n")
    f.write("|_run_no_|_peak_id_|_apex_volume_ml_|_height_|_
        sample_code_imputed_|_stage_imputed_|_n")
    f.write("
        |-----|-----|-----|-----|-----|-----|
        n")

    for _, row in pd.DataFrame(raw_rows).iterrows():
        f.write(f"|_{row['run_no']}|_{row['peak_id']}|_{row['apex_volume_ml']
            '}:.2f}|_{row['height']:.2f}|_{row['sample_code_imputed']}|_{row
            ['stage_imputed']}|_n")

    f.write("\n###CalculatedMetrics\n")
    f.write("|_run_no_|_peak_id_|_apex_volume_ml_|_deconvolved_peak_count_|_
        min_resolution_rs_|_gradient_slope_at_apex_|_sample_code_imputed_|_
        stage_imputed_|_status_|_n")

```

```

f.write("
|-----|-----|-----|-----|-----|
n")

for _, row in pd.DataFrame(calc_rows).iterrows():
    f.write(f"|_{row['run_no']}|_{row['peak_id']}|_{row['apex_volume_ml']:.2f}|_{row['deconvolved_peak_count']}|_{row['min_resolution_rs']:.2f}|_{row['gradient_slope_at_apex']:.4f}|_{row['sample_code_imputed']}|_{row['stage_imputed']}|_{row['status']}|_
|n")

if total_truncated_count > 0:
    f.write(f"\n*Summary_{truncated}_{total_truncated_count}_{additional}_{peaks_detected}*n")

def main():
    df = load_and_preprocess_data()
    df = clip_negative_signals(df)
    df = apply_baseline_correction(df)
    df = impute_metadata(df)

    raw_peaks_df = detect_peaks_derivative(df)

    if not raw_peaks_df.empty:
        calc_peaks_df = deconvolve_peaks(df, raw_peaks_df)
        calc_peaks_df = calculate_gradient_slope(df, calc_peaks_df)
        generate_output(df, raw_peaks_df, calc_peaks_df)
    else:
        with open(OUTPUT_FILE, 'w') as f:
            f.write("No_{peaks_detected}.n")

if __name__ == "__main__":
    main()
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np

# Define paths
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_metrics_raw.md'

# Step 1: Load Data
cols_to_load = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
                'fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise'
                ]

try:
    df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)
except FileNotFoundError:
    raise FileNotFoundError(f"Input_{file}_{INPUT_FILE}_{not_found}.")

# Ensure numeric types
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
df['UV_2_260_mAU'] = pd.to_numeric(df['UV_2_260_mAU'], errors='coerce')
df['fraction_volume_ml_min_precise'] = pd.to_numeric(df['

```

```

    'fraction_volume_ml_min_precise'], errors='coerce')
df['fraction_volume_ml_max_precise'] = pd.to_numeric(df['
    'fraction_volume_ml_max_precise'], errors='coerce')

# Sort by run and volume
df = df.sort_values(by=['run_no', 'volume_ml']).reset_index(drop=True)

def detect_peaks_and_metrics(group):
    """
    Detects peaks within a run group and calculates metrics.
    Optimized to prevent timeouts by replacing while loops with vectorized
    operations.
    """
    vol = group['volume_ml'].values
    uv280 = group['UV_1_280_mAU'].values
    uv260 = group['UV_2_260_mAU'].values

    n = len(uv280)
    if n == 0:
        return pd.DataFrame(columns=['peak_id', 'FWHM', 'Height', 'Area_280', '
            Area_260', 'Ratio_260_280', 'Tailing_Factor'])

    max_sig = np.nanmax(uv280)
    threshold = max_sig * 0.05 if not np.isnan(max_sig) else 0
    baseline = np.nanmin(uv280)
    noise_floor = baseline + (max_sig * 0.01)

    valid_mask = ~np.isnan(uv280)
    left_neighbor = np.roll(uv280, 1)
    right_neighbor = np.roll(uv280, -1)

    peak_mask = (uv280 > left_neighbor) & (uv280 > right_neighbor) & (uv280 >
        threshold) & valid_mask
    peak_mask[0] = False
    peak_mask[-1] = False

    peak_indices = np.where(peak_mask)[0]

    if len(peak_indices) == 0:
        return pd.DataFrame(columns=['peak_id', 'FWHM', 'Height', 'Area_280', '
            Area_260', 'Ratio_260_280', 'Tailing_Factor'])

    run_peaks = []
    peak_counter = 1

    # Pre-compute boolean mask for noise floor to speed up boundary search
    above_floor_mask = uv280 > noise_floor

    for p_idx in peak_indices:
        # Vectorized boundary finding using searchsorted on boolean mask
        # Find left boundary: first index <= p_idx where value drops below
        # noise_floor, then +1
        # We look at the slice up to p_idx
        left_slice = above_floor_mask[:p_idx+1]
        # Find indices where it is False (below floor)
        false_indices = np.where(~left_slice)[0]
        if len(false_indices) > 0:
            # The boundary is the index after the last False before the peak
            last_false = false_indices[-1]

```

```

        left = last_false + 1
    else:
        left = 0

    # Find right boundary: first index >= p_idx where value drops below
    # noise_floor, then -1
    right_slice = above_floor_mask[p_idx:]
    false_indices_right = np.where(~right_slice)[0]
    if len(false_indices_right) > 0:
        # The boundary is the index before the first False after the peak
        first_false_rel = false_indices_right[0]
        right = p_idx + first_false_rel - 1
    else:
        right = n - 1

    # Ensure bounds are valid
    if left < 0: left = 0
    if right >= n: right = n - 1
    if left > right:
        continue

    peak_vol = vol[left:right+1]
    peak_uv280 = uv280[left:right+1]
    peak_uv260 = uv260[left:right+1]

    if len(peak_uv280) == 0:
        continue

    height = np.max(peak_uv280)

    # FWHM
    half_max = height / 2
    half_max_mask = peak_uv280 >= half_max
    if np.any(half_max_mask):
        l_idx = np.argmax(half_max_mask)
        r_idx = len(peak_uv280) - 1 - np.argmax(half_max_mask[::-1])
        fwhm = peak_vol[r_idx] - peak_vol[l_idx]
    else:
        fwhm = 0

    # Area
    area_280 = np.trapz(peak_uv280, peak_vol)
    area_260 = np.trapz(peak_uv260, peak_vol)

    ratio = area_260 / area_280 if area_280 != 0 else 0

    # Tailing Factor
    five_pct = height * 0.05
    five_pct_mask = peak_uv280 >= five_pct
    if np.any(five_pct_mask):
        front_idx = np.argmax(five_pct_mask)
        back_idx = len(peak_uv280) - 1 - np.argmax(five_pct_mask[::-1])
        w_05 = peak_vol[back_idx] - peak_vol[front_idx]
        max_idx_in_seg = np.argmax(peak_uv280)
        f_dist = peak_vol[max_idx_in_seg] - peak_vol[front_idx]

        if f_dist > 0:
            tailing_factor = w_05 / (2 * f_dist)
        else:

```

```

        tailing_factor = 0
    else:
        tailing_factor = 0

    run_peaks.append({
        'peak_id': peak_counter,
        'FWHM': fwhm,
        'Height': height,
        'Area_280': area_280,
        'Area_260': area_260,
        'Ratio_260_280': ratio,
        'Tailing_Factor': tailing_factor
    })
    peak_counter += 1

    return pd.DataFrame(run_peaks)

# Apply peak detection per run
results_list = []
for run_no, group in df.groupby('run_no'):
    peak_df = detect_peaks_and_metrics(group)
    if not peak_df.empty:
        peak_df['run_no'] = run_no
        results_list.append(peak_df)

if not results_list:
    with open(OUTPUT_FILE, 'w') as f:
        f.write("No peaks detected in the dataset.\n")
    exit()

df_all_peaks = pd.concat(results_list, ignore_index=True)

# Step 5: Compute Tailing Factor SD across all runs
run_mean_tail = df_all_peaks.groupby('run_no')['Tailing_Factor'].mean()
tf_mean_global = run_mean_tail.mean()
tf_sd_global = run_mean_tail.std()

tol_lower = tf_mean_global - 2 * tf_sd_global
tol_upper = tf_mean_global + 2 * tf_sd_global

def classify_tail(tf):
    if pd.isna(tf):
        return 'Fail'
    if 0.8 <= tf <= 1.8:
        return 'Pass'
    else:
        if tf < tol_lower or tf > tol_upper:
            return 'Outlier'
        return 'Fail'

df_all_peaks['Tailing_Status'] = df_all_peaks['Tailing_Factor'].apply(
    classify_tail)
df_all_peaks['Tolerance_Band_Lower'] = tol_lower
df_all_peaks['Tolerance_Band_Upper'] = tol_upper

# Pre-compute sorted and truncated DataFrames for each run
grouped_peaks = df_all_peaks.groupby('run_no')
preprocessed_runs = {}
total_truncated_count = 0

```

```

for run_no in sorted(df_all_peaks['run_no'].unique()):
    run_df = grouped_peaks.get_group(run_no)
    run_df = run_df.sort_values(by='Height', ascending=False)
    total_peaks = len(run_df)
    display_count = 20

    if total_peaks > display_count:
        truncated = total_peaks - display_count
        total_truncated_count += truncated
        preprocessed_runs[run_no] = {'full': run_df, 'display': run_df.head(
            display_count), 'truncated': truncated}
    else:
        preprocessed_runs[run_no] = {'full': run_df, 'display': run_df, 'truncated': 0}

# Output Formatting
output_lines = []

# --- Table 1: Raw Detection ---
output_lines.append("###RawDetection")
output_lines.append("Peakdetectioncompletedbasedonlocalmaximaand
    thresholding.")
output_lines.append("")

raw_header_cols = ['run_no', 'peak_id', 'FWHM', 'Height', 'Area_280', 'Area_260',
    'Ratio_260_280', 'Tailing_Factor']
raw_header_str = "|" + "|".join(raw_header_cols) + "|"
raw_sep_str = "|" + "|".join(["---"] * len(raw_header_cols)) + "|"

output_lines.append(raw_header_str)
output_lines.append(raw_sep_str)

for run_no in sorted(preprocessed_runs.keys()):
    data = preprocessed_runs[run_no]
    run_df_display = data['display']
    truncated = data['truncated']

    # Check for missing columns to prevent KeyError
    missing_cols = [col for col in raw_header_cols if col not in run_df_display.
        columns]
    if missing_cols:
        continue

    if truncated > 0:
        output_lines.append(f"|{run_no}|...|...|...|...|...|...|...|...|...|...|...|...|...|...|...|")
        output_lines.append(f"|Summarytruncated:{truncated}additionalpeaks
            detected|")
        output_lines.append("")

    # Ensure all columns exist, fill missing with NaN
    row_data = run_df_display.reindex(columns=raw_header_cols, fill_value=np.nan).
        astype(str)
    float_cols = ['FWHM', 'Height', 'Area_280', 'Area_260', 'Ratio_260_280', '
        Tailing_Factor']
    for col in float_cols:
        if col in row_data.columns:
            row_data[col] = row_data[col].apply(lambda x: f"{float(x):.4f}" if x

```

```

        != 'NaN' and pd.notna(x) else 'NaN')

    for _, row in row_data.iterrows():
        line = "|" + "|".join(row.values) + "|"
        output_lines.append(line)

    output_lines.append("")

# --- Table 2: Calculated Metrics ---
output_lines.append("###CalculatedMetrics")
output_lines.append("")

calc_header_cols = ['run_no', 'peak_id', 'FWHM', 'Height', 'Area_280', 'Area_260',
                    'Ratio_260_280', 'Tailing_Factor', 'Tailing_Status', 'Tolerance_Band_Lower', '
                    Tolerance_Band_Upper']
calc_header_str = "|" + "|".join(calc_header_cols) + "|"
calc_sep_str = "|" + "|".join(["---"] * len(calc_header_cols)) + "|"

output_lines.append(calc_header_str)
output_lines.append(calc_sep_str)

for run_no in sorted(preprocessed_runs.keys()):
    data = preprocessed_runs[run_no]
    run_df_display = data['display']
    truncated = data['truncated']

    # Check for missing columns
    missing_cols = [col for col in calc_header_cols if col not in run_df_display.
                    columns]
    if missing_cols:
        continue

    if truncated > 0:
        output_lines.append(f"|{run_no}|...|...|...|...|...|...|...|...|...|
        |...|...|...|...|")
        output_lines.append(f"|Summary|truncated:{truncated}|additional|peaks|
        detected|")
        output_lines.append("")

    # Ensure all columns exist, fill missing with NaN
    row_data = run_df_display.reindex(columns=calc_header_cols, fill_value=np.nan)
    .astype(str)
    float_cols = ['FWHM', 'Height', 'Area_280', 'Area_260', 'Ratio_260_280', '
    Tailing_Factor', 'Tolerance_Band_Lower', 'Tolerance_Band_Upper']
    for col in float_cols:
        if col in row_data.columns:
            row_data[col] = row_data[col].apply(lambda x: f"{float(x):.4f}" if x
            != 'NaN' and pd.notna(x) else 'NaN')

    for _, row in row_data.iterrows():
        line = "|" + "|".join(row.values) + "|"
        output_lines.append(line)

    output_lines.append("")

# Final summary note
output_lines.append(f"Total|truncated|peaks|across|both|sections:|{
    total_truncated_count}|")

```

```

# Write to file
with open(OUTPUT_FILE, 'w') as f:
    f.write('\n'.join(output_lines))

print(f"Analysis complete. Results saved to {OUTPUT_FILE}")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Step 1: Align peak metrics with fraction boundaries
# Explicitly handle missing fraction_volume_ml_min/max_precise by skipping logic
# for those rows
# We filter out rows with missing values to ensure valid fraction alignment
# Include 'chromatography_stage' in required_cols but handle it carefully in
# dropna
required_cols = ['run_no', 'volume_ml', 'fraction_volume_ml_min_precise', '
    fraction_volume_ml_max_precise',
    'UV_1_280_mAU', 'UV_2_260_mAU', 'Sample_Code', '
    chromatography_stage']

# Check for column presence before selection to implement robust fallbacks
available_cols = [col for col in required_cols if col in df.columns]
if len(available_cols) < len(required_cols):
    missing = set(required_cols) - set(available_cols)
    # Fill missing columns with a default placeholder if they are missing entirely
    for col in missing:
        df[col] = 'Unknown' if col in ['Sample_Code', 'chromatography_stage'] else
            np.nan
    df = df[required_cols].copy()
else:
    df = df[required_cols].copy()

# Ensure numeric types
for col in ['volume_ml', 'fraction_volume_ml_min_precise', '
    fraction_volume_ml_max_precise', 'UV_1_280_mAU', 'UV_2_260_mAU']:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Explicit loop to skip rows with missing fraction volume values and log them
# Replaces df.dropna(subset=...) for these specific columns
rows_to_keep = []
for idx, row in df.iterrows():
    if pd.isna(row['fraction_volume_ml_min_precise']) or pd.isna(row['
        fraction_volume_ml_max_precise']):
        print(f"Skipping row {idx}: Missing fraction volume boundaries.")
    else:
        rows_to_keep.append(idx)
df = df.loc[rows_to_keep].copy()

# Drop rows where other required numeric columns are missing
# Ensure 'chromatography_stage' and 'Sample_Code' are NOT in the dropna subset
numeric_required = [col for col in required_cols if col not in ['Sample_Code', '
    chromatography_stage']]

```



```

df = df.dropna(subset=numeric_required)

# Step 2: Calculate aggregate 260/280 area ratio for each fraction
def calc_fraction_ratio(group):
    group = group.copy()
    # Sort by volume_ml for integration
    group = group.sort_values('volume_ml')

    if len(group) < 2:
        return pd.Series({'fraction_ratio': np.nan})

    # Use np.trapezoid to avoid DeprecationWarning (NumPy 1.24+)
    try:
        area_280 = np.trapezoid(group['UV_1_280_mAU'], group['volume_ml'])
        area_260 = np.trapezoid(group['UV_2_260_mAU'], group['volume_ml'])
    except AttributeError:
        # Fallback for older NumPy versions
        area_280 = np.trapz(group['UV_1_280_mAU'], group['volume_ml'])
        area_260 = np.trapz(group['UV_2_260_mAU'], group['volume_ml'])

    # Check for zero or near-zero area_280 to prevent division errors
    if abs(area_280) < 1e-10:
        frac_ratio = np.nan
    else:
        frac_ratio = area_260 / area_280

    return pd.Series({'fraction_ratio': frac_ratio})

# Group by run and the fraction boundaries
# Optimize Grouping Key Generation: Use tuple for better performance than string
# concat
df['fraction_key'] = list(zip(df['fraction_volume_ml_min_precise'], df['fraction_volume_ml_max_precise']))

# Apply aggregation
fraction_stats = df.groupby(['run_no', 'fraction_key']).apply(calc_fraction_ratio)
fraction_stats.reset_index()
fraction_stats.columns = ['run_no', 'fraction_key', 'fraction_ratio']

# Merge back to original dataframe
df = df.merge(fraction_stats, on=['run_no', 'fraction_key'], how='left')

# Step 3: Identify outliers where ratio deviates >2 SD from run mean
run_stats = df.groupby('run_no')['fraction_ratio'].agg(['mean', 'std']).reset_index()
run_stats.columns = ['run_no', 'run_mean', 'run_std']

df = df.merge(run_stats, on='run_no', how='left')

# Flag outliers with explicit NaN handling for run_std
# Initialize is_outlier to False to handle cases where run_std is NaN
df['is_outlier'] = False

# Check for NaN in run_std and ensure is_outlier remains False in those cases
mask_valid_std = ~df['run_std'].isna()
mask_outlier = (df['fraction_ratio'] > (df['run_mean'] + 2 * df['run_std'])) | \
    (df['fraction_ratio'] < (df['run_mean'] - 2 * df['run_std']))

# Apply logic only where std is valid

```

```

df.loc[mask_valid_std & mask_outlier, 'is_outlier'] = True

# Explicitly cast to boolean and fill NaN with False to prevent unexpected
behavior
df['is_outlier'] = df['is_outlier'].astype(bool).fillna(False)

# Step 4: Detect consecutive outliers using vectorized logic
def detect_consecutive_outliers_vectorized(group):
    group = group.sort_values('volume_ml').copy()
    mask = group['is_outlier'].values

    # Safety check for empty arrays
    if len(mask) == 0:
        group['is_consecutive_outlier'] = []
        return group

    # Identify runs of True values
    # Create a label for each consecutive block
    # 1. Find where the value changes
    changes = np.diff(np.concatenate([[0], mask.astype(int), [0]]))
    starts = np.where(changes == 1)[0]
    ends = np.where(changes == -1)[0] - 1

    # Safety check for empty starts/ends
    if len(starts) == 0:
        group['is_consecutive_outlier'] = np.zeros(len(mask), dtype=bool)
        return group

    # Calculate lengths
    lengths = ends - starts + 1

    # Create a boolean mask for the result
    consecutive_mask = np.zeros(len(mask), dtype=bool)

    # Apply mask only for blocks with length >= 3
    valid_indices = np.where(lengths >= 3)[0]
    for idx in valid_indices:
        s, e = starts[idx], ends[idx]
        consecutive_mask[s:e+1] = True

    group['is_consecutive_outlier'] = consecutive_mask
    return group

df = df.groupby('run_no', group_keys=False).apply(
    detect_consecutive_outliers_vectorized)

# Step 5: Generate consolidated image file
# Pre-compute Run Groups for Plotting (Eliminate Redundant Sorting inside loop)
run_groups = [group.sort_values('volume_ml').reset_index(drop=True) for _, group
               in df.groupby('run_no')]
n_runs = len(run_groups)

if n_runs == 0:
    print("No data to plot.")
    exit()

cols = int(np.ceil(np.sqrt(n_runs)))
rows = int(np.ceil(n_runs / cols))

```

```

fig, axes = plt.subplots(rows, cols, figsize=(15 * cols, 10 * rows))
if n_runs == 1:
    axes = np.array([[axes]])
elif rows == 1:
    axes = axes.reshape(1, -1)
elif cols == 1:
    axes = axes.reshape(-1, 1)

for idx, run_data in enumerate(run_groups):
    row_idx = idx // cols
    col_idx = idx % cols
    ax = axes[row_idx, col_idx]

    # Get run_no safely, handling potential NaN or non-string types
    run_no = run_data['run_no'].iloc[0]
    # Explicit type casting and NaN filling for metadata fields
    if pd.isna(run_no):
        run_no_str = str(run_no).strip()
    else:
        run_no_str = 'Unknown'

    # Plot UV_1_280_mAU
    ax.plot(run_data['volume_ml'], run_data['UV_1_280_mAU'], label='280nm Signal',
            color='blue', linewidth=1)

    # Vectorize Vertical Line Plotting
    # Plot fraction boundaries (start of each fraction)
    unique_starts = run_data['fraction_volume_ml_min_precise'].unique()
    # Fix: Iterate over unique_starts to avoid ValueError with axvline
    if len(unique_starts) > 0:
        for start_val in unique_starts:
            ax.axvline(x=start_val, color='gray', linestyle='--', alpha=0.5)

    # Vectorize Segment Filling
    mask = run_data['is_consecutive_outlier']

    if mask.any():
        # fill_between with 'where' parameter handles disjoint segments
        # efficiently
        ax.fill_between(run_data['volume_ml'], run_data['UV_1_280_mAU'], 0,
                        where=mask, color='red', alpha=0.3, label='Consecutive Outliers')

    ax.set_title(f'Run {run_no_str}: Purity and Consecutive Outlier Analysis')
    ax.set_xlabel('Volume (ml)')
    ax.set_ylabel('UV_280 (mAU)')
    ax.legend(loc='upper right', fontsize='small')
    ax.grid(True, linestyle=':', alpha=0.6)

# Hide unused subplots
for idx in range(n_runs, rows * cols):
    row_idx = idx // cols
    col_idx = idx % cols
    fig.delaxes(axes[row_idx, col_idx])

plt.tight_layout()
plt.savefig('/workspace/purity_outliers_all_runs.png', dpi=150, bbox_inches='tight')
plt.close()

```

```
print("Analysis complete. Output saved to /workspace/purity_outliers_all_runs.png")  
)
```

Listing 3: Code_3